Réaliser une application informatique complète

RAPPORT DE CONCEPTION

Licence Informatique - 3ème Année

COMME Matthieu

FETTAH Sofiane

Année universitaire 2025-2026

1. Introduction **3**
2. Architecture du système **3**
3. Choix Techniques **4**
4. Modélisation métier **5**
4.1 Diagramme de classes métier 5
4.2 Justification des choix de conception 6
5. Protocole de communication **7**
5.1 Architecture des échanges 7
5.2 Format des données et Sérialisation 7
5.3 Spécification des Messages 7
5.3.1 Actions du joueur (incoming) 8
5.3.2 Mises à jour du jeu (outgoing) 8
6. Logique de Jeu **9**
6.1 Cycle de vie d'une partie 9
6.2 Algorithme de gestion du tour 9
6.3 Validation et traitement d'un coup 9
6.4 Gestion des Cartes Spéciales 10
6.5 Recyclage de la pioche 10
6.6. Logique du bot (IA) 10
7. Interface Utilisateur **11**
7.1 Objectifs de l'interface 11
7.2 Écrans et composants principaux 11
7.3 Gestion des interactions (règles côté UI) 13
7.4 Gestion des états et données affichées 13
7.5 Limites connues / évolutions possibles 14
8. Spécification des tests **15**
8.1 Tests unitaires (JUnit 5 & Mockito) 15
8.2. Tests d'Intégration : La validation des flux 15
9. Conclusion **16**

1. Introduction

Ce rapport de conception présente les choix techniques et architecturaux retenus pour la réalisation d'une application web en réseau dans le cadre de la SAE4 **"Réaliser une application informatique complète".**

L'objectif du produit est de fournir un jeu UNO multijoueur en réseau, respectant les règles officielles, les contraintes d'informations cachées et permettant la sauvegarde et la reprise des parties, tout en assurant une expérience fluide et cohérente pour les joueurs.

2. Architecture du système

L'architecture du système repose sur un **modèle à trois niveaux**, fonctionnant de manière **asynchrone** pour garantir une expérience fluide.

Le premier niveau est le **Front-end**. Cette partie gère l'interface visuelle et les interactions avec l'utilisateur.

Le deuxième niveau est le **Back-end**. Le serveur est indépendant et centralise toute la logique de l'application. Il est qualifié d'autoritaire. Il reçoit les intentions des joueurs et vérifie leur validité selon les règles du Uno. Il est le seul à décider de l'état réel de la partie pour éviter les erreurs ou la triche. La communication s'appuie sur le protocole **WebSockets**. Ce tunnel de communication permet d'envoyer des informations en temps réel sans recharger la page.

Le troisième niveau concerne la **gestion des données**. Les informations de l'état du jeu et les paramètres sont stockés dans des fichiers au format **JSON**, facilitant la lecture et la sauvegarde des données.

3. Choix Techniques

Après discussion avec les professeurs, nous avons décidé de revoir nos ambitions à la hausse. Les choix techniques suivants diffèrent donc du cahier des charges rédigé précédemment.

Back-end : Spring Boot (Java 17). Ce choix permet de créer un serveur robuste grâce à son typage fort et sa gestion de la mémoire. Java assure une grande stabilité pour gérer les règles du jeu et les connexions des joueurs.

Front-end: React (JavaScript). Ce framework permet de créer une interface réactive. L'affichage des cartes et du plateau se met à jour automatiquement dès que les données changent.

Communication : WebSockets et STOMP. Pour le temps réel, nous utilisons les WebSockets. Le protocole STOMP est ajouté pour structurer les échanges de messages. Cela permet au serveur d'envoyer des notifications aux joueurs sans qu'ils aient besoin de rafraîchir la page.

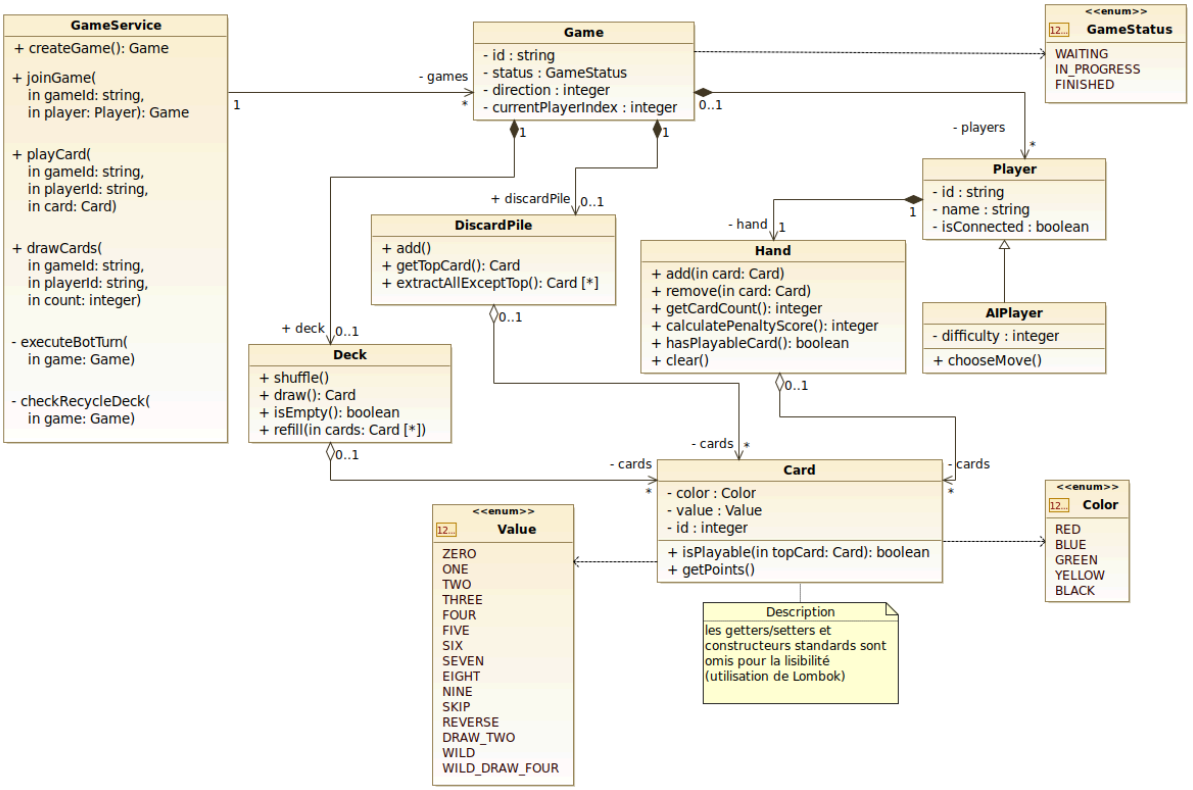
Format de données : JSON. Tous les échanges entre le client et le serveur utilisent le format JSON. C'est un format textuel léger et universel, facile à manipuler en Java comme en JavaScript. Les informations de la partie sont aussi sauvegardées dans ce format.

Outils de développement : IntelliJ IDEA pour le développement du serveur et **VS Code** pour la partie Web. **Git** est utilisé pour synchroniser notre travail en binôme et conserver l'historique des modifications.

4. Modélisation métier

4.1 Diagramme de classes métier

Le diagramme suivant représente l'architecture back-end. Il met en évidence l'organisation des entités et leurs interactions. Comme nous découvrons des nouvelles technologies, ce diagramme peut être sujet à modification.



Plutôt que de laisser une simple liste de cartes dans la classe **Player**, la classe **Hand** isole la logique liée. Le **Deck** (pioche) et la **DiscardPile** (défausse) sont tous les deux des piles de cartes. Mais on les sépare pour gérer de manière autonome leur propre logique.

Player gère l'état du participant. L'héritage vers **AIPlayer** permet d'implémenter des stratégies de jeu sans modifier la structure de base du joueur. **Game** centralise l'état complet d'une partie à un instant T : ordre des joueurs, sens du jeu, piles de cartes…

4.2 Justification des choix de conception

Identification des cartes : pour que le front et le back soient accordés sur l'identité des cartes, nous mettons en place un système d'id. Il existe 108 cartes : 25 par couleur (rouge, vert, bleu et jaune), et 8 noires.

Encapsulation et sécurité : tous les attributs du modèle sont définis en privé. L'état d'une partie ne peut être modifié que par des méthodes contrôlées.

Typage fort via les Enums : l'utilisation d'énumérations pour les couleurs, les valeurs de cartes et le statut de la partie empêche l'introduction de données incohérentes.

Responsabilité unique : nous déléguons la gestion des règles à **GameService**. Les classes du modèle restent simples, tandis qu'il gère les interactions complexes entre les joueurs et les piles de cartes.

Gestion des relations:

- **Composition** : utilisée pour la relation **Player-Hand** car une main n'existe que pour son joueur. Même idée pour les relations avec **Game** : si on supprime la partie, les joueurs, la pioche et la défausse n'existent plus.
- **Agrégation** : utilisée pour les cartes dans **Deck** ou **Hand**, car les objets **Card** sont mobiles et survivent à la destruction de leur conteneur initial.

5. Protocole de communication

Cette partie décrit les mécanismes d'échanges de données entre le client (React) et le serveur (Spring Boot). Le choix du protocole **WebSocket**, complété par la surcouche **STOMP**, a été fait pour permettre une communication bidirectionnelle en temps réel, indispensable pour un jeu multijoueur.

Note importante : les routes, structures JSON et événements décrits ci-dessous sont fournis **à titre indicatif**. Ce protocole constitue une base de travail pour la conception. Des ajustements techniques pourront être apportés lors de la phase d'implémentation finale pour répondre à des contraintes de performance ou de sécurité.

5.1 Architecture des échanges

Nous utilisons un système de **Routing par destination** pour séparer les flux d'informations :

- **les Commandes (client → serveur) :** envoyées sur des destinations privées, elles déclenchent une action logique dans le GameService
- **les Événements (serveur → client) :** diffusés sur des canaux de souscription, ils permettent de mettre à jour l'interface de tous les joueurs simultanément

5.2 Format des données et Sérialisation

Les échanges s'effectuent exclusivement au format **JSON**. La transformation des objets Java en chaînes JSON est automatisée par la bibliothèque **Jackson** intégrée à Spring Boot.

Pour optimiser la bande passante, et par confidentialité, seules les informations nécessaires à l'affichage sont transmises. Ce choix garantit l'absence de triche par lecture du code source côté client.

5.3 Spécification des Messages

Un **DTO** (Data Transfer Object) est un objet dont le seul but est de transporter des données d'un point A à un point B (ici du client vers le serveur). Au lieu d'envoyer un objet entier avec toutes ses informations, on peut transférer une version plus légère avec seulement les données nécessaires. Si le DTO n'est pas conforme, le serveur rejette la requête.

5.3.1 Actions du joueur (incoming)

L'ID du joueur n'est pas envoyé car il est déduit de la session WebSocket pour des raisons de sécurité (éviter l'usurpation d'identité).

Action	**Dest. STOMP**	**Objet DTO (JSON)**	**Rôle**
Rejoindre	/app/join	{ "name": String }	un nouveau joueur entre dans le salon
Jouer	/app/play	{ "gameId: String, "card": Card }	le joueur pose une carte sur la pile
Piocher	/app/draw	{ "gameId": String }	le joueur demande une nouvelle carte
Dire Uno	/app/say-uno	{ "gameId": String }	le joueur signale qu'il ne lui reste qu'une carte
Contre-Uno	/app/uno-out	{ "gameId": String }	le joueur signale qu'un autre joueur a oublié de dire Uno

5.3.2 Mises à jour du jeu (outgoing)

Dès qu'une action est validée par le serveur, l'état modifié est diffusé aux clients concernés. Les canaux /topic sont utilisés pour mettre à jour l'affichage de tous les participants.

Événement	**Canal de diffusion**	**Contenu du message**
Mise à jour	/topic/game/{id}	L'état de la partie (carte visible, tour, nombre de cartes par joueur)
Annonce	/topic/announcements	Information textuelle sur le déroulement (ex: "X a posé un +4")
Erreur	/user/queue/errors	Message privé en cas d'action invalide (ex: "Ce n'est pas votre tour")

6. Logique de Jeu

6.1 Cycle de vie d’une partie

Le déroulement d'une partie suit une machine à états finis simple, passant par trois phases principales :

- **Initialisation (WAITING) :** création de l'objet Game, mélange du deck et distribution des 7 cartes initiales à chaque joueur.
- **En cours (IN_PROGRESS) :** cycle de tours géré par un index de joueur et une valeur de direction.
- **Fin de partie (FINISHED) :** déclenchée dès qu'une Hand devient vide.

6.2 Algorithme de gestion du tour

La gestion de qui doit jouer repose sur deux variables d'état dans la classe Game : **currentPlayerIndex**, l'indice du joueur actif dans la liste et **direction,** une valeur entière (1 ou -1). Pour passer au tour suivant, le service utilise la formule suivante : $nextIndex = (currentIndex + direction) \bmod nbJoueurs$

Cette approche permet de gérer le changement de sens (carte "Inversion") en changeant simplement le signe de la direction.

6.3 Validation et traitement d'un coup

Lorsqu'un message PlayRequest est reçu, le service suit un protocole de vérification strict avant de modifier l'état du jeu :

1. **Vérification du tour :** le serveur compare le playerId de l'expéditeur avec celui attendu par le currentPlayerIndex
2. **Validation de la carte :** le serveur vérifie que le joueur possède bien la carte dans sa main et appelle la méthode card.isPlayableOn(topCard).
3. **Application de l'effet :** si la carte est valide, elle est déplacée vers la défausse et son effet spécial est activé

6.4 Gestion des Cartes Spéciales

Les effets sont traités immédiatement après la pose de la carte :

- **Passer le tour / Inversion :** modification directe de l'index ou de la direction avant de passer au joueur suivant.
- **Cartes de pioche (+2/+4) :** le service appelle la méthode draw() du deck pour le joueur suivant et saute son tour
- **Changement de couleur (Joker)** : le jeu est mis à jour avec une couleur imposée que le prochain joueur devra respecter.

6.5 Recyclage de la pioche

Pour éviter qu'une partie ne s'arrête faute de cartes, un algorithme de recyclage est implémenté. Lorsque la méthode deck.draw() détecte que la pioche est vide :

1. Elle récupère toutes les cartes de la DiscardPile (sauf celle du sommet)
2. Elle les mélange (Collections.shuffle())
3. Elle les réintègre comme nouveau Deck

6.6. Logique du bot (IA)

Dans le cas d'un AIPlayer, la logique de jeu est automatisée. Le service déclenche une méthode qui simule une réflexion. L'IA parcourt sa main pour trouver les cartes jouables, puis en fonction de son niveau :

- **Aléatoire** : joue la première carte jouable
- **Stratège :** priorité aux cartes de forte valeur ou aux cartes spéciales pour gêner l’adversaire

Si aucune carte n'est jouable, elle appelle automatiquement l'action de pioche.

7. Interface Utilisateur

7.1 Objectifs de l'interface

L'interface utilisateur proposera des éléments simples et lisibles, permettant une prise en main rapide du jeu. Les actions principales seront directement accessibles, notamment piocher une carte, jouer une carte et annoncer UNO.

L'interface affichera également les éléments visuels essentiels au déroulement de la partie, tels que la main du joueur, la défausse, la pioche et le sens du jeu. Pour cela nous nous inspirons des jeux UNO disponibles en ligne car ils respectent tous la même logique d'affichage.

Concernant les autres joueurs, leurs mains resteront cachées afin de respecter le principe des informations dissimulées. Seuls leur nom, le nombre de cartes qu'ils possèdent et, le cas échéant, leur avatar seront visibles.

7.2 Écrans et composants principaux

L'interface utilisateur est composée de plusieurs écrans permettant de guider le joueur tout au long de son utilisation de l'application.

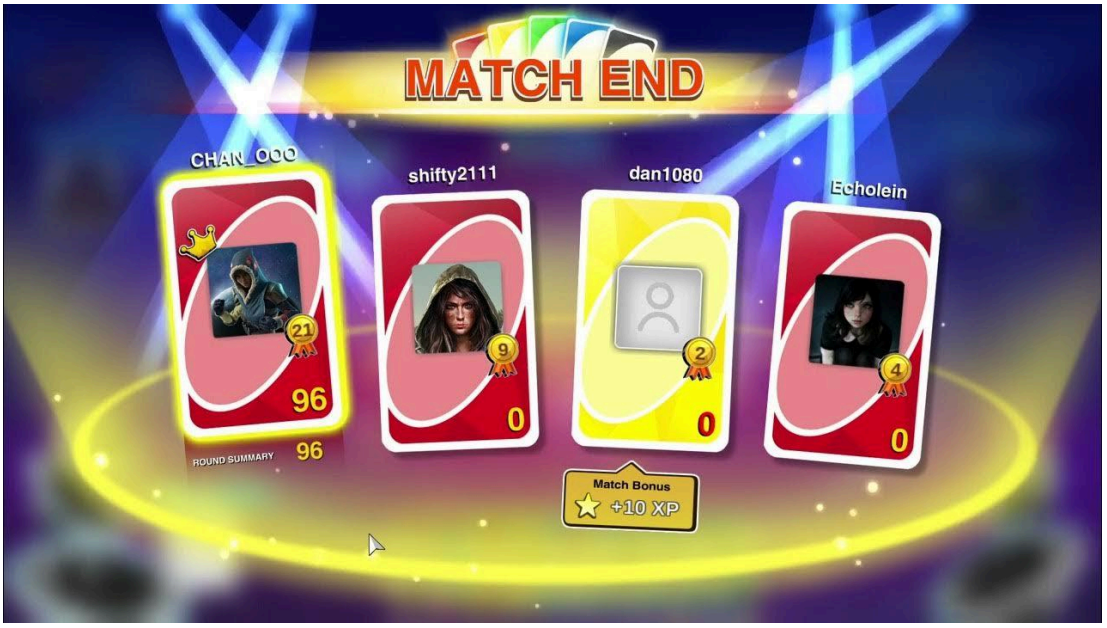
L’écran d’accueil permet :

- Créer une partie local (bots)
- Créer une partie multijoueur.
- Rejoindre une partie multijoueur.
- Saisir son pseudo.
- Lire les règles.
- Paramètre et ainsi modifier les paramètres audio et l’affichage.

Le lobby permet :

- Affiche la liste des joueurs connectés à la ainsi que leur nombre.
- Permet à l’hôte de lancer la partie tandis que les autres joueurs attendent le démarrage.

Un écran de fin de partie est affiché lorsque les conditions de victoire sont atteintes. Il indique le gagnant et met fin aux interactions liées à la partie en cours.



7.3 Gestion des interactions (règles côté UI)

Les interactions de l'utilisateur sont réalisées à travers l'interface graphique et transmises au serveur. Lorsqu'un joueur sélectionne une action, telle que jouer une carte ou piocher, l'intention est envoyée au serveur qui valide l'action en fonction des règles du jeu et de l'état courant de la partie.

Le client contient une logique métier liée aux règles du jeu très simple. Il se contente simplement de comparer la couleur et le numéros de la carte en main avec la défausse si une condition est valide alors la requête est envoyée . Les actions impossibles ou non autorisées sont bloquées ou signalées à l'utilisateur par des messages d'erreur.

Ce fonctionnement garantit la cohérence du jeu, empêche toute triche côté client et assure une synchronisation correcte entre tous les joueurs.

7.4 Gestion des états et données affichées

L'interface utilisateur distingue clairement les informations publiques et les informations privées.

Les informations publiques :

- La carte actuelle de la défausse
- Le joueur actif
- Le sens du jeu
- Le nombre de cartes détenues pour chaque joueur.
- Un timer pour chaque tour de main / Timer depuis le début de la partie.

Les informations privées :

- Le contenu de la main de chaque joueur.
- Le contenu de la pioche.
- Carte reçue lors d'une pioche / pénalités.

Cette séparation permet de garantir la confidentialité des données tout en assurant un affichage cohérent de l'état global de la partie.

7.5 Limites connues / évolutions possibles

L'interface utilisateur repose sur des technologies web qui nous sont nouvelles dans le cadre de ce projet. Leur prise en main progressive a conduit à privilégier une interface fonctionnelle et stable plutôt que l'implémentation de fonctionnalités avancées.

Ainsi, certaines améliorations telles que des animations graphiques complexes, des effets visuels avancés ou une gestion plus poussée de la reconnexion des joueurs ne sont pas intégrées dans la version actuelle de l'application. Le développement s'est volontairement concentré sur les fonctionnalités essentielles garantissant le bon déroulement du jeu et le respect des contraintes du projet.

Ces fonctionnalités avancées pourront être envisagées comme des évolutions futures, si une meilleure maîtrise des technologies utilisées est acquise.

8. Spécification des tests

La qualité et la stabilité du jeu reposent sur une stratégie de tests à plusieurs niveaux. L'objectif est de s'assurer que chaque règle du UNO est respectée et que le serveur reste robuste face à des comportements imprévus. Nous avons adopté une approche structurée pour couvrir l'ensemble de l'application.

8.1 Tests unitaires (JUnit 5 & Mockito)

Le moteur de jeu étant le cœur de l'application, il est couvert par des tests automatisés.

- **Validation des cartes :** tester qu'un "7 Rouge" est jouable sur un "7 Bleu" ou un "9 Rouge", mais pas sur un "3 Vert".
- **Logique du Deck :** vérifier que la pioche diminue bien à chaque retrait et que le recyclage de la défausse se déclenche automatiquement lorsqu'elle est vide.
- **Calcul de l'index :** tester la formule du modulo lors d'un changement de sens pour s'assurer qu'aucun index négatif n'est généré.

8.2. Tests d'Intégration

Si les tests unitaires vérifient la logique isolée, les tests d'intégration s'assurent que les différents composants (Controller, Service, DTO) communiquent correctement entre eux.

- **Le flux de données (End-to-End) :** nous vérifions la boucle complète. Par exemple, lorsqu'un message JSON arrive sur la route /app/play, nous testons qu'il est correctement transformé en **DTO**, traité par le **Service**, et qu'un nouvel état du jeu est bien diffusé sur le canal /topic/game.
- **La cohérence des API :** nous utilisons **@SpringBootTest** pour charger le contexte de l'application et simuler des échanges réels. L'objectif est de garantir que le protocole établi avec le Front-end est respecté : chaque action doit produire la réponse attendue.
- **Persistance JSON :** un test spécifique valide le cycle de **sauvegarde/chargement**. On vérifie l'écriture d'une partie dans un fichier JSON, puis sa lecture, qu'aucune donnée n'est corrompue et permet de reprendre le jeu exactement là où il s'était arrêté.

9. Conclusion

La conception de cette application de UNO multijoueur a permis de transformer un défi amusant en une problématique d'architecture logicielle complexe. En faisant le choix d'un **serveur autoritaire** sous Spring Boot et d'une interface réactive sous React, nous avons posé les bases d'un système robuste, capable de gérer les contraintes du temps réel tout en garantissant l'intégrité des règles du jeu.

Un projet aux ambitions revues à la hausse

Initialement prévu comme une application plus simple, le projet a évolué vers une architecture utilisant les WebSockets et le protocole STOMP. Ce virage technologique nous a permis de répondre à certaines exigences :

- **Sécurité :** grâce à l'utilisation de DTO et d'une validation côté serveur, l'application est protégée contre les tentatives de triche
- **Fluidité :** l'asynchronisme du système assure une expérience de jeu sans rechargement de page, fidèle aux standards des jeux en ligne modernes
- **Intelligence :** l'intégration d'un moteur d'IA, bien que modulaire, offre une profondeur de jeu supplémentaire et une solution pour combler les parties incomplètes

Bilan et perspectives

Ce travail de conception a été l'occasion de mettre en pratique des concepts avancés tels que le **découplage** (via les DTO), l'**encapsulation stricte** et la gestion d'états asynchrones. Si l'interface actuelle se concentre sur l'efficacité fonctionnelle, la structure modulaire de notre code permet d'envisager sereinement des évolutions futures, comme l'ajout de sons, d'animations avancées ou d'un système de classement.

En conclusion, ce projet ne se limite pas à la simple reproduction d'un jeu de cartes. Il représente une immersion complète dans le cycle de vie d'une application.